GOULOT · KIT DE CONSTRUCTION

Mises à jour automatiques

Version du 10 octobre 2026 · texte de référence pour Claude Code : `docs/UPDATES.md`

Goulot se met à jour seul, la nuit. Une version ne s'installe que si l'utilisateur a approuvé sa publication, si sa signature est valide et si elle passe un test de santé ; en cas d'échec, elle est annulée automatiquement.

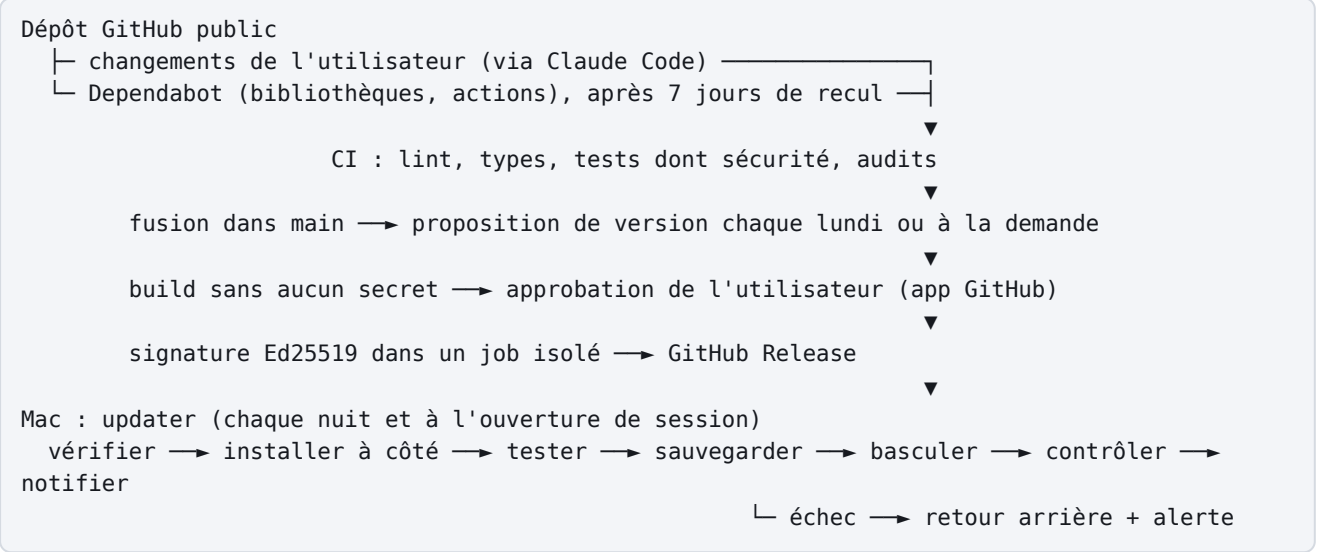
Dépôt : public recommandé

Choix à faire avec l'utilisateur au moment de créer le dépôt, puis à consigner dans `docs/DECISIONS.md`.

- **Public (recommandé).** Le code ne contient ni secret ni donnée personnelle (règle 5 de `CLAUDE.md`). Gratuitement : protection de la branche `main`, environnement `release` avec approbation obligatoire, attestations de provenance, minutes de CI illimitées. L'updater télécharge les releases sans jeton.
- **Privé.** Sur un compte GitHub gratuit, ni protection de branche ni approbation d'environnement. La signature se fait alors sur le Mac, à la demande, et l'updater a besoin d'un jeton en lecture seule. Écarts détaillés en fin de document.

La suite décrit le cas public.

Vue d'ensemble



Côté GitHub

- Branche `main` protégée par un ruleset : CI obligatoire avant fusion, pas de push forcé ni de suppression. L'utilisateur est le seul à pouvoir écrire. Les workflows lancés par des PR extérieures attendent son approbation et ne reçoivent aucun secret.
- `ci.yml` (PR et push) : `uv sync --locked`, `ruff`, `mypy`, `pytest` (dont `tests/security`), `pip-audit`, `bandit`, `npm ci --ignore-scripts`, tests frontend, `npm audit --audit-level=high`

(dépendances de développement comprises : elles tournent pendant le build), build du frontend, `gitleaks`. Permissions : `contents: read`.

- `dependabot.yml` : écosystèmes `uv`, `npm` et `github-actions`, chaque semaine, mises à jour groupées, `cooldown: default-days: 7`. Une nouvelle version n'est proposée qu'après 7 jours de recul, le temps qu'une version piégée soit repérée et retirée ; les correctifs de sécurité ne sont pas retardés.
- Fusion automatique des PR Dependabot de niveau correctif quand la CI est verte (`dependabot-automerge.yml`, qui vérifie que l'auteur est `dependabot[bot]` ; permissions `contents: write` et `pull-requests: write`, sans autre secret) ; les montées mineures et majeures attendent l'utilisateur. Rien n'atteint le Mac sans l'approbation de la release.
- `release.yml` : chaque lundi (cron) ou à la demande (`workflow_dispatch`, ou `goulot dev release`), seulement si `main` a changé depuis la dernière release. Version de correctif incrémentée par défaut ; mineure ou majeure à la demande de l'utilisateur.
  1. **Job `build`, sans aucun secret** (permissions `contents: read`, `id-token: write`, `attestations: write`) : vérifie que le commit est sur `main` (`git merge-base --is-ancestor`), relance les tests, construit `goulot-X.Y.Z.tar.gz` (code Python, `pyproject.toml`, `uv.lock`, frontend compilé, `prompts/`, `schemas/`, `config/` sans secret, migrations), calcule son SHA-256 et sa taille, produit une attestation de provenance (`actions/attest-build-provenance`), et rédige `notes_fr` par script, sans IA : titres des commits depuis la dernière release et bibliothèques changées dans les fichiers de verrouillage. Les notes s'affichent dans le résumé du job ; archive et notes sont publiées comme artefacts.
  2. **Job `sign`, environnement `release`** (approbation obligatoire de l'utilisateur, limité à `main` ; permissions `contents: write`) : récupère les artefacts, vérifie l'attestation (`gh attestation verify` avec le workflow `release.yml` comme signataire attendu), recalcule l'empreinte, écrit `manifest.json` (`name`, `version`, `commit`, `created_at`, `sha256`, `size`, `min_updater_version`, `migrations`, `notes_fr`), le signe en Ed25519 avec `openssl` et le secret d'environnement `GOULOT_SIGNING_KEY`, puis crée le tag et la release avec `gh`. Aucun code tiers n'y tourne : pas d'installation de dépendances, seulement les actions officielles épinglées et les outils du runner.
- La demande d'approbation arrive en notification dans l'app GitHub du téléphone. Elle ouvre l'exécution du workflow, dont le résumé du job `build` affiche `notes_fr` : l'utilisateur les lit avant d'approuver. Sans réponse, la demande expire au bout de 30 jours.
- Toutes les actions sont épinglées par empreinte de commit.

Côté Mac

Emplacements

Élément	Emplacement
Copie de travail de Claude Code	`~/Projects/goulot`, jamais exécutée par les LaunchAgents
Versions installées	`~/Library/Application Support/Goulot/releases/X.Y.Z/`
Version active	lien `~/Library/Application Support/Goulot/current`
Base et sauvegardes	`…/Goulot/data/goulot.sqlite`, `…/Goulot/backups/`
État de l'updater	`…/Goulot/state.json` : plus haute version jamais installée, versions rejetées, échecs consécutifs

Élément	Emplacement
Journaux	`~/Library/Logs/Goulot/`
Lanceur	`~/.local/bin/goulot`, qui exécute la version de `current`
LaunchAgents	`~/Library/LaunchAgents/com.goulot.app.plist` et `com.goulot.updater.plist`, toujours vers `current`

Dossiers en 0700, fichiers en 0600.

Première installation (phase 5)

Pendant les phases 1 à 4, l'app se lance à la main depuis la copie de travail (`uv run goulot start`), sans LaunchAgent. En phase 5, une fois la première release publiée, l'utilisateur lance lui-même dans son Terminal `uv run goulot dev bootstrap`, qui la télécharge, la vérifie avec la clé publique de `goulot/updater/trust.py` et l'installe exactement comme une mise à jour, puis crée le lanceur et, avec l'accord de l'utilisateur, les LaunchAgents. Le code de la copie de travail ne tourne jamais en production.

L'updater

LaunchAgent `com.goulot.updater` : chaque nuit à 4 h 30 et à l'ouverture de session. Réglage dans l'app : automatique (par défaut), me demander, ou désactivé.

1. **Vérifier.** Lire la dernière release du dépôt par l'API GitHub. Télécharger manifeste, signature et archive dans un dossier temporaire en 0700.
2. **Authentifier.** Vérifier la signature avec les seules clés publiques compilées dans `goulot/updater/trust.py` de la version active. Vérifier taille et SHA-256 de l'archive. Exiger une version strictement supérieure à la plus haute version jamais installée, absente des versions rejetées, et un `min_updater_version` compatible.
3. **Installer à côté.** Extraire dans `releases/X.Y.Z/` en refusant chemins absolus, `..`, liens symboliques ou physiques et fichiers spéciaux. Créer l'environnement avec `uv sync --locked --no-dev --no-build` : le projet n'est pas empaqueté (`[tool.uv] package = false`), rien n'est construit depuis les sources, et uv ne télécharge depuis PyPI (`pypi.org`, `files.pythonhosted.org`) que des wheels dont l'empreinte figure dans le `uv.lock` signé. Avec l'envoi des e-mails d'alerte, c'est la seule sortie réseau hors du client HTTP maison.
4. **Tester.** `goulot doctor --release releases/X.Y.Z`, dans un processus séparé : imports, migrations sur une copie de la base, chaîne d'empreintes du registre intacte après migration, chaque champ chiffré toujours déchiffrable avec la clé maîtresse, démarrage de l'app en mémoire avec le client de test ASGI (sans port réseau) et appel de quelques routes avec une session créée dans la copie de la base. Le code de production ne contient aucun moyen de contourner l'authentification.
5. **Sauvegarder.** Copier la base avec l'API de sauvegarde de SQLite, en notant la version qu'elle précède. Cette sauvegarde de bascule est gardée tant que la version précédente reste installée, hors de la rotation quotidienne.
6. **Basculer.** Arrêter le service, appliquer les migrations sur la vraie base, remplacer le lien `current` de façon atomique, redémarrer `com.goulot.app`, mettre à jour `state.json`.
7. **Contrôler.** Si `/healthz` ne répond pas correctement dans les 60 secondes : remettre l'ancien lien, restaurer la sauvegarde, redémarrer, ajouter la version aux rejetées et alerter.
8. **Notifier.** Sur le canal choisi : « Goulot X.Y.Z installé » et `notes_fr`, ou le motif de l'échec.

Règles :

- Conserver les 3 dernières versions installées, les 7 dernières sauvegardes quotidiennes et les sauvegardes de bascule de ces versions.
- Après 2 échecs consécutifs, l'updater se met en pause et alerte l'utilisateur.
- `goulot update --rollback` revient à une version encore installée, restaure la sauvegarde de bascule prise juste avant le passage à la version abandonnée et ajoute celle-ci aux rejetées. Si cette sauvegarde manque, la commande s'arrête avec un message clair, sans rien modifier. Aucun téléchargement d'une version antérieure.
- Un retour arrière, manuel ou automatique, ne perd ni le registre ni les saisies de l'utilisateur : avant de restaurer, l'updater exporte les entrées du registre et les saisies privées (telles quelles, chiffrées) postérieures à la sauvegarde, puis les réinscrit dans l'ordre après restauration ; les empreintes se recalculent à l'identique. Si une réinscription échoue, le retour arrière s'arrête sans rien modifier et alerte l'utilisateur. Seuls les articles collectés entre-temps sont perdus ; la veille en rattrape une partie.
- Migrations additives seulement (nouvelles tables ou colonnes), pour qu'une version précédente relise la base.
- Aucune migration ne modifie le registre des conseils ; une version qui en casserait la chaîne d'empreintes échoue au test de santé.
- Aucune migration ne déchiffre de données privées : les migrations SQL déplacent des colonnes chiffrées telles quelles ; un changement de format des données privées se fait par l'app, coffre ouvert. La clé maîtresse ne change jamais lors d'une mise à jour.
- Tout se passe dans le dossier de l'utilisateur, sans `sudo`.

Clé de signature

- Création unique avec `goulot dev keygen`, lancé par l'utilisateur dans son Terminal, hors du bac à sable de Claude Code : génère la paire Ed25519, envoie la clé privée dans le secret d'environnement `GOULOT_SIGNING_KEY` (`gh secret set --env release`) sans l'afficher, en garde une copie de secours dans un élément de trousseau séparé (`goulot-signing`, hors du service `goulot` de l'app, créé sans application autorisée : toute lecture demande le mot de passe macOS), et inscrit la clé publique dans `goulot/updater/trust.py`.
- Seules les clés de `trust.py`, livré dans les versions signées, font foi ; aucun fichier modifiable sur le disque n'ajoute de clé. Rotation : une version signée par la clé actuelle livre un `trust.py` avec l'ancienne et la nouvelle clé ; les versions suivantes sont signées par la nouvelle, puis l'ancienne est retirée.
- Clé compromise : aucune version signée ne peut l'écarter, puisque l'attaquant pourrait signer lui aussi. Procédure : nouvelle clé, nouvelle release, réinstallation manuelle avec `goulot dev bootstrap`. `goulot panic` affiche cette procédure.

Dépôt privé : écarts

- Pas de job `sign` sur GitHub. `goulot dev release`, lancé sur le Mac avec l'accord de l'utilisateur, attend la fin du job `build`, télécharge l'artefact, vérifie son empreinte par rapport à celle du résumé du workflow, signe avec la clé de l'élément `goulot-signing` (mot de passe macOS demandé) et crée la release avec `gh`.
- L'updater utilise un jeton GitHub à granularité fine, en lecture seule sur ce seul dépôt, stocké dans le trousseau (service `goulot`).

- Pas d'attestation de provenance (réservée aux dépôts publics hors offres Enterprise). Build sur runner Linux pour ménager les minutes gratuites.

Mise à jour des données

- Les règles de veille et la configuration des sources évoluent avec les versions.
- Les chaînes reconstruites ne changent jamais seules : l'app propose de les rafraîchir quand un maillon dépasse 12 mois, et l'utilisateur valide.

Tests obligatoires (phase 5)

`test_update_tampered_rejected` (archive ou manifeste modifiés), `test_untrusted_key_rejected`, `test_update_downgrade_rejected` (y compris sous la plus haute version jamais installée), `test_update_rejected_version_not_reinstalled`, `test_update_path_traversal_rejected`, `test_update_healthcheck_rollback`, `test_update_manual_rollback_restores_backup`, `test_rollback_keeps_ledger_and_private_inputs`, `test_update_pause_after_two_failures`, `test_key_rotation`.